Formalizing Faugère's F_4 Algorithm in Lean 4

Dongwook Cheon

Department of Mathematical Sciences, Seoul National University, Seoul, Republic of Korea

ARTICLE HISTORY
Compiled October 26, 2026

ABSTRACT
We formalize a version of Faugère's F_4 algorithm in Lean 4 using Mathlib's multivariate polynomials and monomial orders. For a finite set of nonzero polynomials over a field in finitely many variables, the formalized procedure returns a Gröbner basis generating the input ideal. The development includes polynomial division with remainder, a formulation of Buchberger's criterion using standard representations, symbolic preprocessing, and Gaussian elimination performed directly on finite sets of polynomials. The correctness proof connects the linear span of the preprocessed polynomials to standard representations over the enlarged basis. Termination follows from a lexicographic well-founded relation combining strict growth of the ideal generated by the current leading monomials with a decrease in the number of unprocessed pairs. The procedure is defined noncomputably; the result verifies its algebraic specification and termination. We describe the proof interfaces between polynomial ideal theory, linear algebra, and the recursive definitions.

KEYWORDS
Formal proof; Gröbner basis; Lean

1. Introduction

Gröbner bases provide algorithms for ideal membership and elimination in polynomial rings over fields. Buchberger's algorithm constructs a Gröbner basis by adjoining nonzero remainders of S-polynomials [1]. Faugère's F_4 algorithm [3] retains this completion principle and processes several pairs in one iteration. Symbolic preprocessing collects suitable monomial multiples of the current generators, after which linear elimination performs the simultaneous reduction. To verify this procedure, one must relate the linear algebra of each batch to the polynomial representations required by Buchberger's criterion.

Gröbner basis algorithms have been formalized in several proof assistants. Théry verified Buchberger's algorithm in Coq [9]. Maletzky and Immler formalized F_4 in Isabelle/HOL within a theory of Gröbner bases for modules, with algorithms that can be translated into executable code [6]. Maletzky subsequently developed an executable formalization of signature-based algorithms [5]. In Lean 4, Guo, Shen, Liu, and Zhi [4] formalized polynomial division, Buchberger's criterion, and the existence and uniqueness of reduced Gröbner bases, including results for infinitely many variables. These developments establish the relevant theory and provide verified algorithms in other sys-

CONTACT Dongwook Cheon. Email: dongwook0826@snu.ac.kr

tems. The present work treats the F_4 procedure directly over Mathlib's polynomial and ideal structures in Lean 4 [8].

Our formalization represents rows as polynomials and performs Gaussian elimination on finite sets of these rows. A fixed monomial order determines the pivot terms. This representation allows the elimination results to be stated in the same polynomial ring as the ideal-theoretic specification, without introducing a separate matrix datatype. The main proof obligation is stronger than preservation of the linear span: processed S-polynomials must admit representations over the enlarged basis whose summands satisfy a leading-monomial bound. Symbolic preprocessing and the properties of the elimination output supply this connection.

1.1. *Main Result*

Let σ be a finite variable type, let K be a field, and fix decidable equality on σ and K. Write $K[\sigma]$ for the multivariate polynomial ring and fix a monomial order $\prec$. For a polynomial set A, write $\langle A \rangle$ for the ideal it generates. For an ideal I, define

$$\mathrm{in}_{\prec}(I) = \langle \mathrm{LM}_{\prec}(f) \mid f \in I,\ f \neq 0 \rangle.$$

Here leading monomials are regarded as monic polynomials when generating an ideal. The following statement summarizes the specification established by the development.

Theorem 1.1. *For every finite set $F \subseteq K[\sigma]$ with $0 \notin F$, the formalized F_4 procedure terminates and returns a finite set of nonzero polynomials G such that*

$$\langle G \rangle = \langle F \rangle, \qquad \langle \mathrm{LM}_{\prec}(g) \mid g \in G \rangle = \mathrm{in}_{\prec}(\langle F \rangle).$$

Thus G is a Gröbner basis of the input ideal.

The procedure is defined by well-founded recursion. Its ideal-preservation property is carried by the field `span_ideal` of its output state, and the theorem `F4_groebner` establishes the Gröbner basis property. The input may be empty; the stated interface requires any zero generators to be removed before applying the procedure.

1.2. *Proof Structure and Scope*

The development first establishes a division procedure and a version of Buchberger's criterion formulated using standard representations. For a nonzero polynomial f, such a representation over G has the form

$$f = \sum_{g \in G} a_g g, \qquad \mathrm{LM}_{\prec}(a_g g) \preceq \mathrm{LM}_{\prec}(f) \quad \text{whenever } a_g g \neq 0.$$

This property is expressed by `reduces_to_zero`. It provides the specification used for processed pairs in the main loop. Polynomial division is used in the development of the criterion; simultaneous reduction in the F_4 loop is performed by symbolic preprocessing and linear elimination.

The elimination routine preserves the K-linear span and returns nonzero monic polynomials with distinct leading monomials, each of which has zero coefficient in the other output polynomials. Combined with the reducer coverage supplied by symbolic

preprocessing, these properties imply that every nonzero polynomial in the preprocessed linear span has a standard representation over the enlarged basis. The lemma `span_ge_symb_proc_red_0` formalizes this implication. It is the step that transfers the result of a batch computation to the invariant for processed S-polynomials.

For termination, let $J(G)$ denote the ideal generated by the leading monomials of the current basis, and let B be the set of unprocessed index pairs. Each iteration either strictly increases $J(G)$ or leaves it unchanged and strictly decreases $|B|$. Since $K[\sigma]$ is Noetherian, strict reverse inclusion of ideals is well founded. The lexicographic combination of this relation with the usual strict order on $\mathbb{N}$ therefore justifies the recursive definition. This is a formal realization of the usual termination argument for completion algorithms, expressed as a relation accepted by Lean's termination checker.

The present procedure is noncomputable. In particular, it uses classical choice to select a nonempty batch of unprocessed pairs and to select reducers during symbolic preprocessing. The verified definition does not expose a pair-selection strategy as a parameter. Its scope is the algebraic correctness and termination of this version of F_4; efficient sparse matrix representations and runtime comparisons remain outside the present development. The source code is available in the project repository [2].

Section 2 specifies the algorithm and its input-output properties. Section 3 develops the polynomial representations and the criterion used in the proof. Section 4 verifies symbolic preprocessing and simultaneous reduction, and Section 5 proves correctness and termination of the main loop. Section 6 reports the formalization and explains how to reproduce the verification. Section 7 discusses limitations and further work.

2. The F4 Procedure and Its Specification

This section specifies the version of Faugère's F_4 algorithm [3] formalized in the development. We describe the data maintained by the main loop and the properties required of symbolic preprocessing and linear elimination. The proofs of these properties and of the main loop specification are given in Sections 4 and 5.

2.1. *Notation and Input Assumptions*

Let K be a field, let σ be a finite type of variables, and write $R = K[\sigma]$. We fix a monomial order $\prec$, with non-strict relation $\preceq$, and use $u \mid v$ for divisibility of monomials. Divisibility and comparison under $\prec$ are distinct relations. For a nonzero polynomial $f \in R$, let $\mathrm{LM}(f)$, $\mathrm{LC}(f)$, and $\mathrm{LT}(f) = \mathrm{LC}(f)\,\mathrm{LM}(f)$ denote its leading monomial, leading coefficient, and leading term, respectively. All leading monomials in this section are taken with respect to $\prec$.

The input is a finite set $F \subseteq R$ such that $0 \notin F$. The empty set is allowed. In Lean, the coefficient type has the instances `Field K` and `DecidableEq K`; the variable type has `Finite` and `DecidableEq` instances. The monomial order is supplied explicitly as an argument. These are the hypotheses of the final correctness theorem.

For a finite set $A \subseteq R$, write $\langle A \rangle$ for its generated ideal and $\mathrm{span}_K(A)$ for its K-linear span. These spans permit polynomial coefficients and scalar coefficients, respectively. Let $\mathrm{Mon}(A)$ be the finite set of monomials with nonzero coefficient in at least one element of A. The ideal generated by the leading monomials of A is denoted by

$$J(A) = \langle \mathrm{LM}(a) \mid a \in A,\ a \neq 0 \rangle. \tag{1}$$

For nonzero $f, g \in R$, put $m = \operatorname{lcm}(\mathrm{LM}(f), \mathrm{LM}(g))$ and define

$$U(f,g) = \frac{m}{\mathrm{LM}(f)} \mathrm{LC}(f)^{-1} f, \qquad V(f,g) = \frac{m}{\mathrm{LM}(g)} \mathrm{LC}(g)^{-1} g. \tag{2}$$

The quotients in (2) are monomial quotients. The ordered pair $(U(f,g), V(f,g))$ is the S-pair used in the implementation, and its difference is the S-polynomial $S(f,g)$. Both components are nonzero and have leading term m, even when $S(f,g) = 0$.

The current generators are stored as a list $G = (g_0, \dots, g_{\ell(G)-1})$ with distinct nonzero entries. We use $\widehat{G}$ for its underlying finite set and define

$$P(G) = \{(i,j) \in \mathbb{N}^2 \mid i < j < \ell(G)\}.$$

The state also records a set $D \subseteq P(G)$ of processed pairs. The unprocessed pairs are $P(G) \setminus D$. In Lean, both pair sets have type `Finset (Fin size × Fin size)`, where `size` is maintained equal to the length of G.

2.2. *Batch Construction and the Main Loop*

Suppose that $P(G) \setminus D$ is nonempty. One iteration selects a nonempty subset C of these pairs and forms

$$H_0 = \bigcup_{(i,j) \in C} \{U(g_i, g_j), V(g_i, g_j)\}. \tag{3}$$

Thus both components of each selected S-pair belong to H_0, and $S(g_i, g_j) \in \mathrm{span}_K(H_0)$ for every $(i,j) \in C$.

Symbolic preprocessing enlarges H_0 to a finite set H of nonzero polynomials. Each element of H is a nonzero scalar multiple of a monomial multiple of an element of $\widehat{G}$. Consequently, $H_0 \subseteq H \subseteq \langle \widehat{G} \rangle$. It also satisfies the following reducer coverage property: for every $u \in \mathrm{Mon}(H)$,

$$\begin{aligned} &(\exists g \in \widehat{G},\ \mathrm{LM}(g) \mid u) \quad \Longrightarrow \\ &\qquad \exists g \in \widehat{G},\ \exists t \text{ a monomial}, \quad tg \in H \ \text{ and } \ \mathrm{LM}(tg) = u. \end{aligned} \tag{4}$$

The procedure starts with no monomials marked as processed. At each step it examines the greatest unprocessed monomial occurring in the current rows. If that monomial is divisible by $\mathrm{LM}(g)$ for some $g \in \widehat{G}$, it adds the corresponding monomial multiple of g. The process continues until all monomials occurring in the rows have been processed.

Linear elimination then returns a finite set N of nonzero monic polynomials satisfying

$$\mathrm{span}_K(N) = \mathrm{span}_K(H), \qquad \mathrm{Mon}(N) = \mathrm{Mon}(H). \tag{5}$$

Its output has distinct leading monomials, and the coefficient of $\mathrm{LM}(n)$ in n' is zero whenever $n, n' \in N$ and $n \neq n'$. The monomials, ordered decreasingly by $\prec$, serve as column labels for these row operations. The implementation performs the operations directly on polynomials rather than constructing a separate matrix datatype.

Algorithm 1 The formalized F_4 procedure

Input: a finite set $F \subseteq R$ with $0 \notin F$.
Output: a Gröbner basis of $\langle F \rangle$.

1. $G \leftarrow \mathrm{list}(F)$; $D \leftarrow \emptyset$.
2. **while** $D \neq P(G)$ **do**
3. Choose $\emptyset \neq C \subseteq P(G) \setminus D$.
4. $H_0 \leftarrow \bigcup_{(i,j)\in C} \{U(g_i, g_j), V(g_i, g_j)\}$.
5. $H \leftarrow \mathrm{SymbolicPreprocess}(\widehat{G}, H_0)$.
6. $N \leftarrow \mathrm{GaussianEliminate}(H)$.
7. $N' \leftarrow \{n \in N \mid \forall h \in H,\ \mathrm{LM}(h) \nmid \mathrm{LM}(n)\}$.
8. $m \leftarrow \ell(G)$.
9. $G \leftarrow G +\!\!+ \mathrm{list}(N')$.
10. $D \leftarrow \iota_{m,\ell(G)}(D \cup C)$.
11. **end while**
12. **return** $\widehat{G}$.

The rows to be added to the current generators are

$$N' = \{n \in N \mid \text{for every } h \in H,\ \mathrm{LM}(h) \nmid \mathrm{LM}(n)\}. \tag{6}$$

The test in (6) uses divisibility by leading monomials of the preprocessed rows. It implies that $\mathrm{LM}(n) \notin J(\widehat{G})$ for each $n \in N'$. Indeed, $\mathrm{LM}(n) \in \mathrm{Mon}(H)$ by (5). If some $\mathrm{LM}(g)$, $g \in \widehat{G}$, divided $\mathrm{LM}(n)$, then (4) would provide a row in H with leading monomial equal to $\mathrm{LM}(n)$, contradicting (6). The divisibility criterion for membership in a monomial ideal gives the stated conclusion. Furthermore, every row in N' belongs to $\langle \widehat{G} \rangle$ by span preservation and the inclusion $H \subseteq \langle \widehat{G} \rangle$.

Algorithm 1 gives the main loop. Here $\mathrm{list}(A)$ denotes a listing without repetitions of a finite set A, and $+\!\!+$ denotes list concatenation. If $m \leq m'$, let $\iota_{m,m'}$ denote the inclusion of pairs of indices smaller than m into pairs of indices smaller than m'. It leaves their numerical values unchanged.

Appending new rows preserves the positions of all existing entries of G. The implementation rebuilds the full set $P(G)$ for the extended list and maps $D \cup C$ into the new index type. In particular, pairs involving new rows are initially unprocessed. If $N' = \emptyset$, the generators remain unchanged and at least one additional pair is marked as processed.

The displayed choice of C corresponds to a classical choice in the Lean definition. Reducer selection during symbolic preprocessing is also made classically. The current definition does not take a pair-selection strategy as a parameter. It retains all pairs until they are processed; no pair discarding criterion is used in this loop.

2.3. *Output Specification and Lean Interface*

The required output is a finite set A of nonzero polynomials such that

$$\langle A \rangle = \langle F \rangle, \qquad J(A) = \mathrm{in}_{\prec}(\langle F \rangle), \tag{7}$$

where $\mathrm{in}_{\prec}(I)$ denotes the ideal generated by leading monomials of the nonzero elements of an ideal I. Thus A is a Gröbner basis of the input ideal. The specification does not require it to be reduced or minimal.

Two invariants connect Algorithm 1 to this specification. First, $\langle \widehat{G} \rangle = \langle F \rangle$ throughout the loop. Second, for each $(i, j) \in D$ with $s = S(g_i, g_j) \neq 0$, there exist coefficients $a_g \in R$ such that

$$s = \sum_{g \in \widehat{G}} a_g g, \qquad \mathrm{LM}(a_g g) \preceq \mathrm{LM}(s) \quad \text{if } a_g g \neq 0. \tag{8}$$

The condition is vacuous when $S(g_i, g_j) = 0$. Upon termination, $D = P(G)$, so the second invariant applies to all distinct pairs of current generators. The standard-representation form of Buchberger's criterion then gives the Gröbner basis property. Section 3 establishes that criterion, Section 4 proves the properties of batch reduction, and Section 5 verifies the invariants and termination of the loop.

In Lean, `F4 mo F hF` returns an element of `F4Struct mo (Ideal.span F)`. If this state is denoted by `s`, then `s.G` is its list of generators and `s.G.toFinset` is the finite set corresponding to A. The field `s.span_ideal` states `Ideal.span F = Ideal.span s.G.toFinset`. The final theorem has the following statement, with its proof omitted here:

```
theorem F4_groebner {σ K : Type*}
  [Finite σ] [DecidableEq σ] [Field K] [DecidableEq K]
  (mo : MonomialOrder σ)
  (F : Finset (MvPolynomial σ K)) (hF : 0 ∉ F) :
    let f4F := F4 mo F hF
    is_groebner mo f4F.G.toFinset (Ideal.span F)
```

The predicate `is_groebner` states containment in the specified ideal and equality of the corresponding leading-monomial ideals. Together with `span_ideal`, the theorem therefore yields (7). Termination is established in the well-founded recursive definition of `F4`; the lemma `F4_full_proc` identifies its terminal condition as equality of the full and processed pair sets. The procedure is defined noncomputably, so this specification concerns the abstract procedure and does not assert executable performance.

3. Polynomial Representations and the Correctness Criterion

We retain the assumptions and notation of Section 2. In the mathematical statements below, A denotes a finite set of polynomials; the list of current generators in the main loop remains denoted by G. This section describes the representation of polynomials and leading monomials, specifies the division procedure, and establishes the form of Buchberger's criterion used for processed pairs.

3.1. *Exponent Vectors and Leading Monomials*

Let $E = \sigma \to_0 \mathbb{N}$ be the type of finitely supported exponent vectors. An exponent $\alpha \in E$ represents the monomial x^α, and addition in E corresponds to multiplication of monomials. Mathlib's `MvPolynomial` σ `K` represents polynomials by finitely supported coefficients on E. The expression `MvPolynomial.monomial` α c denotes cx^α, and the support of a polynomial is the finite set of exponents with nonzero coefficient.

The type E has a pointwise order. For $\alpha, \beta \in E$,

$$\alpha \le \beta \quad \Longleftrightarrow \quad x^{\alpha} \mid x^{\beta}. \tag{9}$$

To use a monomial order on the same exponents, Mathlib provides an ordered type synonym `mo.syn` and an additive equivalence `mo.toSyn` : $E \simeq_{+}$ `mo.syn`. Thus the code compares exponents in E for divisibility and compares their images under `mo.toSyn` for the monomial order. This distinction is needed both for testing divisibility and for proving strict decrease in recursive definitions.

For a nonzero polynomial f with $\mathrm{LM}(f) = x^{\alpha}$, the function `leading_monomial'` returns α. Its nonzeroness argument ensures that the support is nonempty, so its maximum can be taken in the specified monomial order. The total function `leading_monomial` instead returns an element of `WithBot` E: it returns the leading exponent for nonzero inputs and $\bot$ for zero. The value $\bot$ is distinct from the zero exponent, which represents the monomial 1.

For mathematical statements involving possibly zero polynomials, write $\mathrm{LM}_{\bot}(0) = \bot$ and $\mathrm{LM}_{\bot}(f) = \mathrm{LM}(f)$ when $f \neq 0$, with $\bot$ below every monomial. The corresponding ordered value in Lean is obtained by applying `WithBot.map mo.toSyn` to `leading_monomial mo f`. For example, the development proves

$$\mathrm{LM}(fg) = \mathrm{LM}(f)\,\mathrm{LM}(g) \quad (f, g \neq 0), \qquad \mathrm{LM}_{\bot}(f+g) \preceq \max\{\mathrm{LM}_{\bot}(f), \mathrm{LM}_{\bot}(g)\}. \tag{10}$$

These properties control the leading monomials of products and sums in the division and criterion proofs. The leading-monomial operations return exponents; the monomials appearing in (10) are their associated monic polynomials.

3.2. *Monomial Ideals and the Gröbner Basis Predicate*

For a set $M \subseteq E$, the definition `monomial_ideal K M` denotes the ideal generated by the monic polynomials x^{μ}, $\mu \in M$. The membership results `mon_mem_moni_iff` and `mem_moni_iff` state, respectively, that

$$x^{\nu} \in \langle x^{\mu} \mid \mu \in M \rangle \quad \Longleftrightarrow \quad \exists \mu \in M,\ \mu \le \nu, \tag{11}$$

$$f \in \langle x^{\mu} \mid \mu \in M \rangle \quad \Longleftrightarrow \quad \forall \nu \in \mathrm{supp}(f),\ \exists \mu \in M,\ \mu \le \nu. \tag{12}$$

The comparisons on the right are pointwise comparisons of exponents. In particular, (11) converts exclusion of a leading monomial from $J(A)$ into the divisibility condition used to select new rows in Section 2.

For a finite set A, `leading_monomials_fin` collects the leading exponents of its nonzero elements. The set-valued definition `leading_monomials` serves the same purpose for an arbitrary polynomial set, including the underlying set of an ideal. Zero contributes no exponent to either collection. These definitions allow $J(A)$ and $\mathrm{in}_{\prec}(I)$ to use the same monomial-ideal construction even though I is generally infinite.

The predicate for a Gröbner basis is defined as follows. In the displayed Lean declarations, the finite-set variable `G` corresponds to the mathematical set A.

```
def is_groebner {σ K : Type*} [DecidableEq σ] [Field K]
  (mo : MonomialOrder σ) (G : Finset (MvPolynomial σ K))
  (I : Ideal (MvPolynomial σ K)) : Prop :=
  ↑G ⊆ I.carrier ∧
```

```
monomial_ideal K (leading_monomials mo I.carrier) =
monomial_ideal K (leading_monomials_fin mo G)
```

It asserts $A \subseteq I$ and $\mathrm{in}_{\prec}(I) = J(A)$. For $0 \notin A$, the lemma `ideal_of_gb_eq_span` derives $I = \langle A \rangle$ from this predicate, using the division results below. Thus ideal generation is a consequence of the stated definition.

3.3. *Division and Zero Remainders*

Fix a list $L = (d_0, \dots, d_{s-1})$ of distinct nonzero polynomials. Division of $f \in R$ by L maintains a quotient list $(q_0, \dots, q_{s-1})$, a remainder r, and a polynomial p still to be processed. The structure `MvPolyDivStruct` includes the identity

$$f = \sum_{i=0}^{s-1} d_i q_i + r + p, \tag{13}$$

together with the bounds

$$\mathrm{LM}_{\perp}(d_i q_i) \preceq \mathrm{LM}_{\perp}(f), \qquad \mathrm{LM}_{\perp}(r) \preceq \mathrm{LM}_{\perp}(f), \qquad \mathrm{LM}_{\perp}(p) \preceq \mathrm{LM}_{\perp}(f). \tag{14}$$

Every monomial of r is required to be indivisible by each $\mathrm{LM}(d_i)$. The structure also records that the quotient and divisor lists have the same length.

Initially all quotients and r are zero and $p = f$. If $p \neq 0$, the step function searches for the first d_i whose leading monomial divides $\mathrm{LM}(p)$. When such an index exists, put $t = \mathrm{LT}(p)/\mathrm{LT}(d_i)$ and replace (q_i, p) by $(q_i + t, p - t d_i)$. If there is no such index, replace (r, p) by $(r + \mathrm{LT}(p), p - \mathrm{LT}(p))$. Each assignment uses the values before the step. In both cases the new value of $\mathrm{LM}_{\perp}(p)$ is strictly smaller. The well-founded order on `WithBot mo.syn` therefore justifies the recursive definition `mvpoly_division_rec`.

The lemma `mvpoly_division_p_eq_0` states that the terminal value of p is zero. Consequently, `mvpoly_division_repn` gives

$$f = \sum_{i=0}^{s-1} d_i q_i + r, \qquad \forall u \in \mathrm{Mon}(\{r\}),\ \forall i < s,\ \mathrm{LM}(d_i) \nmid u, \tag{15}$$

and the bounds (14) remain available in the returned state. The decomposition, bounds, and remainder condition are distinct parts of the specification.

Let $\rho_L(f)$ denote the remainder of this procedure, and let $\mathcal{L}(A)$ be the set of listings of A without repetitions. For a finite set A with $0 \notin A$, `remainder_zero` means that $\rho_L(f) = 0$ for some $L \in \mathcal{L}(A)$. The predicate `remainder_zero_ord_indep` requires this for every $L \in \mathcal{L}(A)$. For a general divisor set, these are different conditions. A zero remainder implies $f \in \langle A \rangle$ by (15).

If A is a Gröbner basis of I, the converse holds for every listing. Indeed, when $f \in I$, the remainder in (15) also belongs to I. If it were nonzero, its leading monomial would belong to $\mathrm{in}_{\prec}(I) = J(A)$, contradicting the remainder condition by (11). This proves

$$\begin{aligned} f \in I \quad &\iff \quad \exists L \in \mathcal{L}(A),\ \rho_L(f) = 0 \\ &\iff \quad \forall L \in \mathcal{L}(A),\ \rho_L(f) = 0. \end{aligned} \tag{16}$$

The two quantified formulations are formalized by `mem_ideal_iff_gb_remainder_zero`

and `mem_ideal_iff_gb_remainder_zero_ord_indep`. The division procedure itself does not require the divisors to form a Gröbner basis, and its declaration does not require a finite variable type.

3.4. *Standard Representations and Buchberger's Criterion*

For $f \neq 0$, a standard representation of f over A consists of coefficients $a_g \in R$ such that

$$f = \sum_{g \in A} a_g g, \qquad \mathrm{LM}(a_g g) \preceq \mathrm{LM}(f) \quad \text{whenever } a_g g \neq 0. \tag{17}$$

This is the definition of `reduces_to_zero`. Despite its name, it specifies a polynomial representation rather than a sequence of reduction steps. It is the condition carried by the processed-pair invariant in Section 2.

The lemma `red_0_of_remainder_0` derives (17) from a zero division remainder: the quotients provide the coefficients, and the summand bounds in (14) give the required leading-monomial inequalities. A standard representation does not determine the remainder for a prescribed ordering of divisors. For example, over $\mathbb{Q}[x, y]$ with lexicographic order $x \succ y$, take $A = \{x - y, x\}$ and $f = x$. The representation $f = 1 \cdot x$ is standard. Division by $(x - y, x)$ gives remainder y, whereas division by $(x, x - y)$ gives remainder zero.

The following form of Buchberger's criterion [1] connects these conditions.

Theorem 3.1 (Buchberger's criterion)**.** *Let $A \subseteq R$ be finite with $0 \notin A$. The following are equivalent.*

(1) A is a Gröbner basis of $\langle A \rangle$.
(2) For every $f, g \in A$, the polynomial $S(f, g)$ has zero remainder for some listing of A without repetitions.
(3) Every nonzero $S(f, g)$, $f, g \in A$, has a standard representation over A as in (17)*.*

In the second condition the listing may depend on the pair. When the conditions hold, (16) implies zero remainder for every listing. Equal pairs impose no additional requirement, since $S(f, f) = 0$. The bound in the third condition is $\mathrm{LM}(S(f, g))$ itself; for a nonzero S-polynomial this is strictly smaller than $\mathrm{lcm}(\mathrm{LM}(f), \mathrm{LM}(g))$.

Proof. The first condition implies the second because each S-polynomial lies in $\langle A \rangle$, and division by a Gröbner basis detects ideal membership. The second implies the third by the division bounds.

For the converse, the case $A = \emptyset$ is immediate. Let $0 \neq h \in \langle A \rangle$ and choose a representation $h = \sum_{g \in A} b_g g$ for which

$$\delta = \max\{\mathrm{LM}(b_g g) \mid g \in A,\ b_g g \neq 0\}$$

is minimal in the monomial order. Such a representation exists by well-foundedness. The leading-monomial bound for a sum gives $\mathrm{LM}(h) \preceq \delta$.

Suppose that $\mathrm{LM}(h) \prec \delta$. For the summands attaining δ, split b_g into its leading term and its remaining terms. The corresponding leading multiples of the g have common leading monomial δ, and their coefficients at δ sum to zero. The cancellation identity

formalized by `syzygy_lemma`, together with the scaling identity `mul_term_spoly`, expresses their sum as a K-linear combination of polynomials

$$t_{g,g'} S(g, g'), \qquad t_{g,g'} \operatorname{lcm}(\mathrm{LM}(g), \mathrm{LM}(g')) = \delta,$$

where each $t_{g,g'}$ is a monomial. Substitute a standard representation for each nonzero $S(g, g')$. All resulting summands have leading monomial strictly below δ, by the third condition and multiplicativity of the monomial order. The terms left after removing the leading terms of the b_g also lie below δ, as do the original summands that did not attain it. Regrouping by generators gives a representation of h with a smaller maximal leading monomial, contradicting the choice of δ.

Hence $\mathrm{LM}(h) = \delta$. Some summand $b_g g$ attains this maximum, so $\mathrm{LM}(g) \mid \mathrm{LM}(h)$ by (10). This holds for every nonzero $h \in \langle A \rangle$, and (11) yields $\mathrm{in}_{\prec}(\langle A \rangle) = J(A)$. Thus A is a Gröbner basis. □

The implications in this proof are organized as `buchberger_tfae_1`, `buchberger_tfae_2`, and `buchberger_tfae_3`. The theorem `buchberger` combines the first two conditions. The form used by the F_4 loop is `refined_buchberger`, whose statement is reproduced below with the proof omitted.

```
theorem refined_buchberger {σ K : Type*}
  [Finite σ] [DecidableEq σ] [Field K] [DecidableEq K]
  (mo : MonomialOrder σ)
  (G : Finset (MvPolynomial σ K)) (hG : 0 ∉ G) :
    is_groebner mo G (Ideal.span G) ↔ every_S_poly_red_0 mo G hG
```

Finally, standard representations are preserved when generators are added: `red_0_on_extension` extends the coefficient family by zero. The lemma `red_0_iff_neg_red_0` handles reversal of an S-pair. These properties allow the main loop to retain the conditions for processed pairs while enlarging its generator list. To establish the same condition for newly processed pairs, Section 4 uses the properties of symbolic preprocessing and linear elimination.

4. Symbolic Preprocessing and Simultaneous Reduction

Section 3 identifies standard representations as the condition required for processed pairs. We now establish this condition for a batch of S-polynomials. Throughout this section, $A = \widehat{G}$ is the finite set underlying the current generator list, and H_0 contains the two components of each selected S-pair, as specified in Section 2. In particular, $0 \notin A$, $0 \notin H_0$, and every element of H_0 is a scalar monomial multiple of an element of A.

The argument has two parts. Symbolic preprocessing supplies a reducer in the preprocessed row set for every reducible monomial occurring in that set. Linear elimination then supplies a row with the same leading monomial as any nonzero polynomial in its linear span. Together, these properties permit induction on the leading monomial, yielding standard representations over the enlarged generator set. All linear spans in this section are over K; $\langle A \rangle$ continues to denote a polynomial ideal.

4.1. *Symbolic Preprocessing and Reducer Coverage*

Write $H = \mathrm{SP}(A, H_0)$ for the output of symbolic preprocessing. Besides preserving the initial rows, the procedure maintains

$$H_0 \subseteq H \subseteq \{ctg \mid g \in A,\ c \in K,\ t \text{ a monomial}\}, \qquad 0 \notin H. \tag{18}$$

Here and below a monomial is monic. Thus $H \subseteq \langle A \rangle$. Preprocessing may enlarge $\mathrm{span}_K(H_0)$; the linear span that elimination must preserve is $\mathrm{span}_K(H)$.

The additional output condition is

$$\begin{aligned} u \in \mathrm{Mon}(H), \quad \exists g \in A,\ \mathrm{LM}(g) \mid u \quad \Longrightarrow \quad & \exists g \in A,\ \exists t \text{ a monomial}, \\ & tg \in H \quad \text{and} \quad \mathrm{LM}(tg) = u. \end{aligned} \tag{19}$$

This condition concerns every monomial in H, including the terms below the leading monomials of the initial rows. It ensures that a reduction of such a monomial by A can be carried out within $\mathrm{span}_K(H)$.

Algorithm 2 describes the procedure. The set T records monomials already considered and is separate from the set of processed index pairs in the main loop.

Algorithm 2 Symbolic preprocessing

Input: finite sets A, H_0 satisfying the conditions above.

1. $H \leftarrow H_0$; $T \leftarrow \emptyset$.
2. **while** $\mathrm{Mon}(H) \setminus T \neq \emptyset$ **do**
3. $\quad u \leftarrow \max_{\prec}(\mathrm{Mon}(H) \setminus T)$.
4. $\quad T \leftarrow T \cup \{u\}$.
5. $\quad$ If some $g \in A$ satisfies $\mathrm{LM}(g) \mid u$, choose one such g and set $H \leftarrow H \cup \{(u/\mathrm{LM}(g))g\}$.
6. **end while**
7. **return** H.

The choice of g is classical, as in the implementation. Its leading coefficient is not normalized at this stage: the inserted multiplier is the monic monomial $u/\mathrm{LM}(g)$. Hence the inserted row has leading monomial u, and each of its other monomials is strictly smaller than u.

The structure `SymbProcStruct` records the intermediate sets H, T as `H` and `done_mons`. The containment and nonzeroness conditions in (18) hold at each step. Its field `div_then_cont_mult` records (19) for the monomials in T, and `done_sub_H` records $T \subseteq \mathrm{Mon}(H)$.

Termination uses the most recently considered monomial, stored in `last_mon`. Initially this value is $\top$ in `WithTop mo.syn`. After processing u, every unprocessed monomial is strictly smaller than u: this holds for previously pending monomials by maximality, and for newly introduced monomials by the leading-monomial property of the inserted row. The field `nd_lt_lmon` records this invariant. Thus successive values of `last_mon` strictly decrease in a well-founded order. The number of pending monomials need not decrease, since a new row can introduce additional terms. Nor is it necessary that the monomials below the initial maximum form a finite set; for example, $y^k \prec x$ for every k in lexicographic order with $x \succ y$.

At termination, `symbolic_preprocess_done_mons` establishes $T = \mathrm{Mon}(H)$. Consequently, the condition recorded for T yields (19) for the whole row set.

4.2. *Elimination on Polynomial Rows*

The elimination routine acts directly on finite sets of polynomials. Let $M = \mathrm{Mon}(H)$ and $V = \mathrm{span}_K(H)$. The structure `GEStruct` maintains a set S_I of rows still to be processed and a set S_O of output rows. Initially $(S_I, S_O) = (H, \emptyset)$. The fields `monset_fixed` and `span_V` assert

$$\mathrm{Mon}(S_I \cup S_O) = M, \qquad \mathrm{span}_K(S_I \cup S_O) = V. \tag{20}$$

Neither set contains zero. Every leading monomial in S_I is smaller than every leading monomial in S_O. The rows in S_O are monic, have distinct leading monomials, and have their pivot monomials absent from all other rows in $S_I \cup S_O$.

If $S_I \neq \emptyset$, choose $p \in S_I$ with maximal leading monomial $u = \mathrm{LM}(p)$ and put $\overline{p} = \mathrm{LC}(p)^{-1}p$. For a polynomial q, let

$$E_p(q) = q - [u]q\,\overline{p}, \tag{21}$$

where $[u]q$ denotes the coefficient of u in q. The step replaces the two row sets by

$$\begin{aligned} S_I' &= \{E_p(q) \mid q \in S_I \setminus \{p\}\} \setminus \{0\}, \\ S_O' &= \{E_p(q) \mid q \in S_O\} \cup \{\overline{p}\}. \end{aligned} \tag{22}$$

In particular, the new pivot is eliminated from previously processed rows as well as from the remaining rows. Previously established pivot coefficients are unchanged, because p has zero coefficient at those monomials.

The row operations preserve the linear span: each old row is recovered from its new value by adding a scalar multiple of the retained pivot. The same observation gives both inclusions for the union of supports in (20). The maximality of $\mathrm{LM}(p)$ ensures that all remaining input leading monomials are smaller than u. The number $|S_I|$ strictly decreases; removal of zero rows and merging of equal rows can make the decrease greater than one. This measure justifies `gaussian_elim_rec`.

The terminal state has $S_I = \emptyset$. Write $N = S_O = \mathrm{GE}(H)$ for its output row set. Then

$$\mathrm{span}_K(N) = V, \qquad \mathrm{Mon}(N) = \mathrm{Mon}(H), \qquad 0 \notin N, \tag{23}$$

and the pivot properties are

$$\begin{aligned} &\mathrm{LC}(n) = 1 && (n \in N), \\ &\mathrm{LM}(n) \neq \mathrm{LM}(m), \quad [\mathrm{LM}(n)]m = 0 && (n, m \in N,\ n \neq m). \end{aligned} \tag{24}$$

If the columns are indexed by M in decreasing monomial order and the output rows are ordered by their leading monomials, these are the pivot properties of a reduced row echelon form. These properties are proved directly for polynomial rows.

Lemma 4.1 (Leading monomials in the row span)**.** *For every nonzero $f \in V$, there exists $n \in N$ such that $\mathrm{LM}(f) = \mathrm{LM}(n)$.*

Proof. Write $f = \sum_{n \in N} c_n n$ with $c_n \in K$. Monicity and pivot separation in (24) give

$$c_n = [\mathrm{LM}(n)]f \qquad (n \in N). \tag{25}$$

Put $u = \mathrm{LM}(f)$. Since $[u]f \neq 0$, some summand satisfies $c_n[u]n \neq 0$. The condition $[u]n \neq 0$ gives $u \preceq \mathrm{LM}(n)$. Since $c_n \neq 0$, equation (25) gives $\mathrm{LM}(n) \preceq u$. Hence $u = \mathrm{LM}(n)$. □

This is the lemma `mem_span_ge_exists_same_lm_mem`. It identifies the leading monomials that may occur in V; preservation of the linear span alone would not provide this identification for an arbitrary row set.

4.3. *From the Row Span to Standard Representations*

As in Section 2, retain the rows

$$N' = \{n \in N \mid \forall h \in H,\ \mathrm{LM}(h) \nmid \mathrm{LM}(n)\}. \tag{26}$$

The test is divisibility by the leading monomials of H. For the output rows, reducer coverage makes it equivalent to exclusion from $J(A)$:

$$n \in N' \quad \Longleftrightarrow \quad \mathrm{LM}(n) \notin J(A) \qquad (n \in N). \tag{27}$$

Indeed, each $h \in H$ is a nonzero scalar monomial multiple of some $g \in A$, so $\mathrm{LM}(h) \mid \mathrm{LM}(n)$ implies $\mathrm{LM}(g) \mid \mathrm{LM}(n)$. Conversely, if $\mathrm{LM}(g) \mid \mathrm{LM}(n)$ for some $g \in A$, then $\mathrm{LM}(n) \in \mathrm{Mon}(H)$ by (23), and (19) supplies a row in H with that same leading monomial. The monomial-ideal membership criterion of Section 3 now gives (27). The use of coverage here is restricted to monomials occurring in H.

Theorem 4.2 (Standard representations after a batch)**.** *Let A, H_0 be finite sets satisfying the conditions at the beginning of this section, and let $H = \mathrm{SP}(A, H_0)$, $N = \mathrm{GE}(H)$, and N' be given by* (26)*. Every nonzero $f \in \mathrm{span}_K(H)$ has a standard representation over $B = A \cup N'$. Explicitly, there are coefficients $a_b \in R$ such that*

$$f = \sum_{b \in B} a_b b, \qquad \mathrm{LM}(a_b b) \preceq \mathrm{LM}(f) \quad \textit{whenever } a_b b \neq 0. \tag{28}$$

Proof. We use well-founded induction on $u = \mathrm{LM}(f)$. By Lemma 4.1, choose $n \in N$ with $\mathrm{LM}(n) = u$. There are two cases.

If $n \in N'$, set $q = \mathrm{LC}(f)n$. The row n is monic, so $\mathrm{LT}(q) = \mathrm{LT}(f)$. Also $q \in V$, and therefore $r = f - q \in V$. If $r \neq 0$, cancellation of the leading term gives $\mathrm{LM}(r) \prec u$, and the induction hypothesis supplies a standard representation of r over B. Adding the summand $\mathrm{LC}(f)n$ gives (28). If $r = 0$, that summand alone gives the required representation.

If $n \notin N'$, some $h \in H$ satisfies $\mathrm{LM}(h) \mid u$. By (18), this implies $\mathrm{LM}(g) \mid u$ for some $g \in A$. Furthermore, $u = \mathrm{LM}(n) \in \mathrm{Mon}(H)$ by (23). Reducer coverage (19) therefore supplies $g \in A$ and a monomial t such that $tg \in H$ and $\mathrm{LM}(tg) = u$. Put

$$q = \frac{\mathrm{LC}(f)}{\mathrm{LC}(g)}\, tg, \qquad r = f - q.$$

Then $\mathrm{LT}(q) = \mathrm{LT}(f)$, and $q \in V$ because $tg \in H$. Thus $r \in V$ and either $r = 0$ or $\mathrm{LM}(r) \prec u$. In the latter case apply the induction hypothesis to r and add the summand

q, whose generator g belongs to $A \subseteq B$. The case $r = 0$ again uses the single summand q.

In both cases, coefficients attached to the same generator are combined. The leading-monomial bound for a sum preserves the required bound under this regrouping: the summands coming from r lie strictly below u, and the added summand has leading monomial u. □

The second case explains the role of symbolic preprocessing in the induction. Divisibility by a current leading monomial supplies a reducer in the ideal $\langle A \rangle$; coverage supplies one in the particular linear space V on which the induction is carried out. Subtracting an arbitrary polynomial multiple of an old generator need not stay in V. In the implementation, `span_ge_symb_proc_red_0_induction` carries out this induction, using `mul_basis_add_red_0` to reconstruct the representation after cancellation.

The theorem is exposed as `span_ge_symb_proc_red_0`. Its statement is reproduced below, with line breaks adjusted and the proof omitted. Here `G` denotes A, `L` denotes H_0, and `sp_L.H` denotes H. The comparison $\leq$ in the filter is the pointwise exponent comparison for divisibility.

```
lemma span_ge_symb_proc_red_0 {σ K : Type*}
  [Finite σ] [DecidableEq σ] [Field K] [DecidableEq K]
  (mo : MonomialOrder σ)
  (G : Finset (MvPolynomial σ K)) (hG : 0 ∉ G)
  (L : Finset (MvPolynomial σ K)) (hL : 0 ∉ L)
  (hGL : ↑L ⊆ {mg | ∃ g ∈ G, ∃ α : σ →₀ ℕ, ∃ c : K,
    mg = g * MvPolynomial.monomial α c}) :
  let sp_L := symbolic_preprocess mo G hG L hL hGL
  let ge_L' := gaussian_elim mo sp_L.H
  let N := ge_L'.SO
  let N' := N.filter (fun n => ∀ l ∈ sp_L.H,
    ¬leading_monomial mo l ≤ leading_monomial mo n)
  let V : Submodule K (MvPolynomial σ K) := Submodule.span K sp_L.H
  ∀ f ∈ V, (f_ne_0 : f ≠ 0) → reduces_to_zero mo f f_ne_0 (G ∪ N')
```

4.4. *An Example of a Batch*

Work in $\mathbb{Q}[x, y]$ with lexicographic order $x \succ y$, and let

$$A = \{g_0, g_1, g_2\}, \qquad g_0 = x^2 - y, \quad g_1 = xy - 1, \quad g_2 = y^2 - 1.$$

Consider a batch consisting of the pairs (g_0, g_1) and (g_1, g_2). The initial rows are

$$\begin{aligned} h_1 &= yg_0 = x^2y - y^2, \quad h_2 = xg_1 = x^2y - x, \\ h_3 &= yg_1 = xy^2 - y, \quad h_4 = xg_2 = xy^2 - x. \end{aligned} \tag{29}$$

At x^2y and xy^2, either available reducer adds a row already in H_0. The monomial x is irreducible by A. At y^2, preprocessing adds $h_5 = g_2 = y^2 - 1$, introducing the monomial 1. Neither y nor 1 is reducible. Thus $H = \{h_1, \dots, h_5\}$ and

$$\mathrm{Mon}(H) = \{x^2y, xy^2, x, y^2, y, 1\}, \qquad x^2y \succ xy^2 \succ x \succ y^2 \succ y \succ 1.$$

The added row lies outside $\mathrm{span}_{\mathbb{Q}}(H_0)$, since every initial row has zero constant coefficient.

Using the displayed order for the columns and the order $h_1, \ldots, h_5$ for the input rows, the coefficient matrix and its reduced form are

$$\begin{bmatrix} 1 & 0 & 0 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & -1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 1 & -1 \end{bmatrix}. \tag{30}$$

Consequently,

$$N = \{x^2y - 1, xy^2 - 1, x - 1, y^2 - 1, y - 1\}, \qquad N' = \{x - 1, y - 1\}.$$

The filter discards the rows with leading monomials x^2y, xy^2, and y^2, since these are leading monomials of rows in H. It retains $x - 1$ and $y - 1$ because neither x nor y is divisible by those leading monomials. This agrees with $J(A) = \langle x^2, xy, y^2 \rangle$ and $J(A \cup N') = \langle x, y \rangle$.

The selected S-polynomials have the standard representations

$$S(g_0, g_1) = x - y^2 = (x - 1) - g_2,$$
$$S(g_1, g_2) = x - y = (x - 1) - (y - 1).$$

Every nonzero summand has leading monomial at most x, the leading monomial of either S-polynomial. The first representation uses an old generator whose row was added during preprocessing, whereas the second uses two retained output rows.

4.5. *Consequences for the Main Loop*

Each selected S-polynomial is the difference of its two initial components and hence belongs to $\mathrm{span}_K(H_0) \subseteq V$. Theorem 4.2 therefore establishes the processed-pair condition of Section 3 for every nonzero selected S-polynomial over $A \cup N'$. Pairs with zero S-polynomial satisfy the condition directly. Previously established standard representations persist by `red_0_on_extension`.

Every row of N' belongs to $V \subseteq \langle A \rangle$, so

$$\langle A \cup N' \rangle = \langle A \rangle. \tag{31}$$

If $N' \neq \emptyset$, equation (27) also gives

$$J(A) \subsetneq J(A \cup N'). \tag{32}$$

In particular, $N' \cap A = \emptyset$, so every retained row is a new generator. These are the batch properties used in Section 5: ideal preservation, standard representations for the newly processed pairs, and strict growth of the leading-monomial ideal whenever new generators are appended. Section 5 combines them with the update of the pair indices to establish the loop invariant and the termination measure.

5. Correctness and Termination of the Main Loop

Section 4 establishes the properties of one batch: the retained rows preserve the generated ideal, the selected S-polynomials acquire standard representations over the enlarged generator set, and every retained row has leading monomial outside the previous leading-monomial ideal. We now use these properties to verify the main loop. The proof must also account for changes in the index type when generators are appended and for the fact that the number of unprocessed pairs can increase.

5.1. *The Loop State and Its Invariants*

Fix $I = \langle F \rangle$, where $F \subseteq R$ is the finite input set and $0 \notin F$. A loop state consists of a generator list $G = (g_0, \dots, g_{r-1})$, its length r, the full pair set

$$P = P(G) = \{(i, j) \mid 0 \le i < j < r\},$$

and a set $D \subseteq P$ of processed pairs. Write $A = \widehat{G}$. The structure `F4Struct mo I` records the following conditions.

(1) The entries of G are distinct and nonzero.
(2) $\langle A \rangle = I$.
(3) For every $(i, j) \in D$, if $S(g_i, g_j) \neq 0$, then $S(g_i, g_j)$ has a standard representation over A.

The second and third conditions are the fields `span_ideal` and `sat_buchberger`. The length equality, the inclusion $D \subseteq P$, and the specification of P are also fields of the structure. Thus a state carries the algebraic invariants together with the data required to index its generators.

In Lean, the pair sets have type `Finset (Fin r × Fin r)`. The field `i_pairs_lt` characterizes membership by the strict inequality of the two index values. The field `i_pairs_card` records

$$|P| = \binom{r}{2} = \frac{r(r-1)}{2}. \tag{33}$$

This identity is supplied by `pair_set_card`, proved for finite subsets of a linearly ordered type. For $r < 2$, the pair set is empty.

The definition `F4_init` takes $G = \mathrm{list}(F)$ and $D = \emptyset$. A listing of a finite set has distinct entries, and $0 \notin F$ gives nonzeroness. The generated ideal is I, and the processed-pair condition is vacuous. Consequently, initialization produces an element of `F4Struct mo I`.

5.2. *Preservation Under One Iteration*

Suppose that $D \neq P$. Since $D \subseteq P$, the set $P \setminus D$ is nonempty. The definition `F4_step` chooses a nonempty subset $C \subseteq P \setminus D$ and forms the initial row set H_0 from the two components of each selected S-pair. Let H, N, and N' be the preprocessed rows, the elimination output, and the retained rows, respectively, as in Section 4. Put $m = |N'|$.

The updated list and pair sets are

$$\begin{aligned} G^+ &= G +\!\!+ \operatorname{list}(N'), \\ P^+ &= P(G^+), \qquad D^+ = \iota_{r,r+m}(D \cup C). \end{aligned} \tag{34}$$

Here $\iota_{r,r+m}$ is the index-pair embedding introduced in Section 2. The implementation applies `Fin.castAdd m` to each coordinate. This preserves the numerical values and the inequality between the indices. It follows that $D^+ \subseteq P^+$, and the entries at those indices are the same polynomials as before the update. Every pair involving a newly appended entry is initially unprocessed.

For $n \in N'$, Section 4 gives $n \neq 0$ and $\mathrm{LM}(n) \notin J(A)$. In particular, $n \notin A$, since the leading monomial of every element of A belongs to $J(A)$. Thus $N' \cap A = \emptyset$. The appended list has no repetitions, and its entries are distinct from those of G. This proves distinctness and nonzeroness for G^+, with length $r + m$.

The same batch results give $N' \subseteq \operatorname{span}_K(H) \subseteq \langle A \rangle$. Since $A \subseteq \widehat{G^+} = A \cup N'$, we obtain

$$\langle \widehat{G^+} \rangle = \langle A \cup N' \rangle = \langle A \rangle = I. \tag{35}$$

This supplies the field `span_ideal` of the new state.

To verify `sat_buchberger`, take a pair in D^+ and pull it back along $\iota_{r,r+m}$. If its preimage belongs to D, its nonzero S-polynomial already has a standard representation over A. The lemma `red_0_on_extension` extends the coefficient family by zero on N', preserving that representation over $A \cup N'$.

If the preimage belongs to C, both components $U(g_i, g_j)$ and $V(g_i, g_j)$ lie in $H_0 \subseteq H$. Hence

$$S(g_i, g_j) = U(g_i, g_j) - V(g_i, g_j) \in \operatorname{span}_K(H).$$

For a nonzero S-polynomial, the batch theorem of Section 4 supplies a standard representation over $A \cup N'$. A zero S-polynomial imposes no additional condition. The preservation of the old list entries identifies these polynomials with the S-polynomials indexed by D^+. These arguments construct the returned `F4Struct mo I` in `F4_step`; invariant preservation is part of the definition's return type.

5.3. *A Well-Founded Measure for the Loop*

Set

$$b(s) = |P(G) \setminus D|, \qquad \mu(s) = \big(J(\widehat{G}), b(s)\big). \tag{36}$$

Appending generators can increase $b(s)$. More precisely, the embedding in (34) is injective, $C \cap D = \emptyset$, and the new pairs consist of rm pairs with one old and one new index and $\binom{m}{2}$ pairs among the new entries. Therefore

$$b(s^+) = b(s) - |C| + rm + \binom{m}{2}. \tag{37}$$

Here $|C| \le b(s)$ because $C \subseteq P(G) \setminus D$, so the difference is nonnegative. This identity explains why the pending-pair count is used only after comparing the leading-monomial ideals.

On $\mathrm{Ideal}(R) \times \mathbb{N}$, define

$$(J', b') \lhd (J, b) \quad \Longleftrightarrow \quad J \subsetneq J' \ \text{ or } \ (J' = J \ \text{ and } \ b' < b). \tag{38}$$

Thus strict growth of the first component counts as a decrease in $\lhd$, regardless of the second component.

Lemma 5.1 (Progress of the main loop)**.** *For every nonterminal state s, the state s^+ returned by one iteration satisfies $\mu(s^+) \lhd \mu(s)$.*

Proof. If $N' \neq \emptyset$, Section 4 gives $J(\widehat{G}) \subsetneq J(\widehat{G^+})$. The first clause of (38) applies.

If $N' = \emptyset$, then $m = 0$ and $G^+ = G$. Under the natural identification of the index types, $P^+ = P$ and $D^+ = D \cup C$. Equation (37) becomes $b(s^+) = b(s) - |C| < b(s)$, since C is nonempty. The leading-monomial ideal is unchanged, so the second clause applies. □

The formal counterpart is `f4s_moni_ipcard_lex_decr`. Its statement is reproduced below, with line breaks adjusted and the proof omitted. The function `f4s_lex_prod` is the measure μ in (36). The relation `(· > ·)` on ideals places the larger successor ideal before its predecessor.

```
lemma f4s_moni_ipcard_lex_decr {σ K : Type*}
  [Finite σ] [DecidableEq σ] [Field K] [DecidableEq K]
  (mo : MonomialOrder σ) (I : Ideal (MvPolynomial σ K))
  (f4s : F4Struct mo I)
  (full_proc : ¬f4s.i_pairs = f4s.i_pairs_proc) :
  let f4s' := F4_step mo I f4s full_proc
  Prod.Lex (· > ·) (· < ·)
    (f4s_lex_prod mo I f4s') (f4s_lex_prod mo I f4s)
```

Because K is a field and σ is finite, the polynomial ring $R = K[\sigma]$ is Noetherian. The ascending chain condition on its ideals makes reverse strict inclusion well founded. Together with the usual well-founded order on $\mathbb{N}$, this yields well-foundedness of $\lhd$. In Lean, the instance `imv_nat_wf` constructs this lexicographic relation from the corresponding Mathlib instances. The finite-variable hypothesis is used here to obtain the Noetherian property.

Consequently, the recursion

$$\mathrm{Loop}(s) = \begin{cases} s, & D = P, \\ \mathrm{Loop}(\mathrm{Step}(s)), & D \neq P \end{cases} \tag{39}$$

is justified by well-founded recursion on $\mu(s)$. The definition `F4_rec` uses (36) in its `termination_by` clause and Lemma 5.1 in its decrease proof. Thus it defines a total function from `F4Struct mo I` to itself.

The stopping condition of the returned state is proved separately. The lemma `F4_rec_full_proc` uses induction on the same well-founded measure: in the terminal branch the condition is already satisfied; in the recursive branch it follows from the induction hypothesis at the strictly smaller successor state. The wrapper `F4` applies `F4_rec` to the

initial state, and `F4_full_proc` gives

$$D_* = P(G_*) \tag{40}$$

for its output state. In particular, no fuel parameter or a priori bound on the number of iterations is required.

5.4. *Correctness of the Terminal State*

Let G_* be the list returned by `F4`, and put $A_* = \widehat{G_*}$. Since the returned value has type `F4Struct mo I`, its entries are distinct and nonzero, and

$$\langle A_* \rangle = I = \langle F \rangle. \tag{41}$$

By (40), the field `sat_buchberger` applies to every index pair $i < j$.

To apply the criterion of Section 3 to all $f, g \in A_*$, choose their indices i, j in G_*. If $i = j$, then $f = g$ and $S(f, g) = 0$. If $i < j$, the processed-pair invariant gives a standard representation for every nonzero $S(f, g)$. If $j < i$, it gives a standard representation for $S(g, f)$ whenever this polynomial is nonzero. The identity

$$S(f, g) = -S(g, f)$$

then supplies one for $S(f, g)$: multiply the coefficient family by -1. The leading-monomial bounds are unchanged. This last step is formalized by `red_0_iff_neg_red_0`, together with the identity `S_poly_neg_of_swap`.

Hence every nonzero S-polynomial of elements of A_* has a standard representation over A_*. The theorem `refined_buchberger` implies

$$J(A_*) = \mathrm{in}_{\prec}(\langle A_* \rangle) = \mathrm{in}_{\prec}(\langle F \rangle). \tag{42}$$

Together with (41), this proves the main result of Section 1. The implementation combines these steps in `F4_groebner`, whose conclusion is `is_groebner mo f4F.G.toFinset (Ideal.span F)`. Ideal generation remains available as the field `f4F.span_ideal`.

The empty input is covered by the same definitions. If $|F| \leq 1$, there are no index pairs, so the initial state already satisfies the stopping condition; the criterion still applies.

6. Formalization Report and Reproducibility

We record the organization of the Lean development and the outcomes of a build and an inspection of axiom dependencies in the author's project environment.

6.1. *Modules and Recursive State Structures*

The development has five main modules. `MonomialIdeal` supplies leading-monomial operations and monomial-ideal membership results. `DivisionAlgorithm` defines division with its decomposition and leading-monomial bounds, and `GroebnerBasis` establishes

the standard-representation criterion. `GaussianElim` implements elimination on polynomial rows. The module `FaugereF4.FaugereF4` defines symbolic preprocessing, combines the batch operations, and proves correctness and termination of the main loop. The root module `FaugereF4` imports these five modules.

The recursive procedures use a common organization: their state structures contain both the evolving data and proofs of the properties needed at the next call. Table 1 summarizes the four state structures. The fields listed there have different roles. Some relate the current data to a fixed input, such as the division identity or preservation of the generated ideal. Others constrain the unfinished part of a computation, such as the ordering of unprocessed monomials or the pivot conditions. Their role in recursion is to make these properties available when constructing the successor state.

Table 1. Recursive states, their principal invariants, and termination measures.

Structure	Principal invariants	Measure
`MvPolyDivStruct`	Decomposition of the input, bounds on summands, and irreducibility of the accumulated remainder	$\mathrm{LM}_{\perp}(p)$
`SymbProcStruct`	Containment of the rows, reducer coverage on the processed monomials, and a bound on the remaining monomials	`last_mon`
`GEStruct`	Preservation of the linear span and support, with separated monic pivots	$\lvert S_I \rvert$
`F4Struct`	Preservation of the ideal and standard representations for processed pairs	$(J(\widehat{G}), \lvert P(G) \setminus D \rvert)$

A recursive call therefore receives a state satisfying the same invariants as its input. Terminal properties are then established by separate lemmas: the remaining dividend is zero, all preprocessing monomials have been considered, the elimination input is empty, or all index pairs have been processed. The distinction between invariant preservation and satisfaction of the stopping condition is used throughout the development.

6.2. *Recorded Environment and Build*

Table 2 gives the versions reported by the command logs. The status queries for both the project and Mathlib reported no changes.

Table 2. Environment recorded for the build and axiom audit.

Component	Recorded value
Lean	`4.27.0-rc1`, release build
Lean commit	`2fcce7258eeb6e324366bc25f9058293b04b7547`
Target	`x86_64-w64-windows-gnu`
Lake	`5.0.0-src+2fcce72`
Project commit	`c1670fd7b5792be362e5128afb45b37999b32d1b`
Mathlib commit	`32d24245c7a12ded17325299fd41d412022cd3fe`

The project commit includes the implementation, configuration, `PaperAudit.lean`, and its saved output.

The recorded commands were

```
lake build
lake env lean PaperAudit.lean
```

Both returned exit code 0. The build log reported successful completion. The audit file imports the root module, prints the definition of `is_groebner`, checks the types of the main declarations, and queries their axiom dependencies. In particular, the printed types of `F4`, `F4_full_proc`, and `F4_groebner` agree with the interfaces described in Sections 2 and 5.

6.3. *Axiom Dependencies*

The command `#print axioms` reports the transitive axiom dependencies of a declaration [7]. The audit applied it to eleven declarations. These include the procedure `F4`, its correctness theorem `F4_groebner`, the completion lemmas `F4_full_proc` and `F4_rec_full_proc`, the progress lemma `f4s_moni_ipcard_lex_decr`, and the well-founded relation `imv_nat_wf`. The remaining targets were `span_ge_symb_proc_red_0`, `mem_span_ge_exists_same_lm_mem`, `gaussian_elim_SI_empty`, `symbolic_preprocess_done_mons`, and `refined_buchberger`.

Every query reported the same three dependencies:

`propext`, `Classical.choice`, `Quot.sound`.

For example, the two principal queries returned

```
'F4' depends on axioms: [propext, Classical.choice, Quot.sound]
'F4_groebner' depends on axioms: [propext, Classical.choice, Quot.sound]
```

No `sorryAx` or additional axiom was reported for any of the eleven targets. The three reported dependencies are the standard principles of propositional extensionality, classical choice, and quotient soundness [7]. The result concerns the transitive dependencies of the audited declarations. It does not assert that every unrelated declaration in the imported libraries was separately inspected.

The query for `F4` also records the dependencies of the construction returning its state, whose fields include ideal preservation. The correctness theorem and the returned state's proof fields thus have distinct interfaces, both represented in the audit.

6.4. *Reproducing the Checks*

Check out the project revision in Table 2 from the source repository [2]. Its configuration uses the Mathlib path `../.modules/mathlib4-v4.27.0-rc1`. Place a Mathlib checkout at the recorded revision in that directory. The committed `lean-toolchain`, `lakefile.toml`, and `lake-manifest.json` specify the toolchain and dependencies. The manifest fixes the resolved revisions of the Git dependencies, including `checkdecls` and `doc-gen4`.

With these checkouts in place, run the two commands above from the project directory. The evidence archive retains their outputs and exit codes, the version and Git-status queries, the configuration files, and the audit script.

The rational example in Section 4 was also checked independently by exact arithmetic. This checks the displayed rows, matrix reduction, filter, and standard-representation identities. It illustrates the batch specification; it is not an execution or a performance measurement of the noncomputable Lean definition.

7. Discussion

7.1. *The Algebraic Content of the Verification*

The batch theorem of Section 4 connects linear algebra and ideal theory. Preservation of the row span establishes ideal membership of the output, but the processed-pair invariant requires control of each summand in a polynomial representation. Reducer coverage and the pivot properties supply that control. The resulting theorem applies to every nonzero element of the preprocessed linear span, and in particular to the selected S-polynomials.

Two features of the argument explain the specifications used in the development. First, preprocessing must provide coverage for all monomials occurring in the rows. In the induction proving the batch theorem, this ensures that leading-term cancellation remains in the same linear space. Second, the filter on retained rows is a divisibility condition. Combined with coverage, it identifies leading monomials outside the previous leading-monomial ideal and hence supplies the strict growth used for termination.

7.2. *Organization of the Recursive Proofs*

The development required identifying the facts needed at successive recursive calls. Packaging the data and loop invariants in one structure gave a uniform way to express this requirement. Each step returns the same state type, retaining the properties needed by the next call. The invariant fields also support the proof that the termination measure decreases.

This organization creates substantial obligations inside state constructions. Updating a generator list changes the types of its finite indices, so the proof must identify old list entries after the index embedding. Likewise, an intermediate preprocessing state records coverage only on the monomials already considered; the stopping-condition lemma is needed before that coverage can be used on the whole support. These obligations concern different mathematical statements, even when their Lean proofs occur in the same definition.

The present code contains long definitions and repeated setup. For example, `F4_step` constructs the selected batch and the embedding of pair indices, and `f4s_moni_ipcard_lex_decr` reconstructs those objects when proving progress. Factoring out a shared batch construction, together with lemmas about its returned data and the index embedding, would remove this duplication. The state structures expose the properties such a refactoring would have to preserve.

7.3. *Scope and Further Work*

The verified procedure is defined noncomputably. The current implementation chooses batches and reducers classically and does not expose a selection strategy as a parameter. Well-founded recursion over a Noetherian polynomial ring in finitely many variables establishes termination, with no executable complexity bound. The output specification is a Gröbner basis generating the input ideal. It does not require a reduced or minimal basis.

An executable refinement would require explicit choices and effective representations for the relevant finite data and coefficient operations. A strategy interface would also need contracts ensuring that each batch is a nonempty subset of the pending pairs and that each selected reducer has the required leading-monomial divisibility. The existing

proof uses these properties, but correctness for a new parameterized implementation would require a corresponding refinement proof.

A further extension is to replace the finite sets of polynomial rows by a matrix representation suitable for sparse elimination. Such a change would require a correspondence between coefficient rows and polynomials, preservation of the linear span, and the pivot properties used in Section 4. The standard-representation theorem gives the algebraic specification that these elimination results must support.

8. Acknowledgement

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2025-00520280).

9. Disclosure

In the preparation of this work, the author used ChatGPT-6.1 Sol in order to improve readability and language. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

[1] B. Buchberger, *Bruno buchberger's phd thesis 1965: An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal*, Journal of Symbolic Computation 41 (2006), pp. 475–511. Available at `https://www.sciencedirect.com/science/article/pii/S0747717105001483`, Logic, Mathematics and Computer Science: Interactions in honor of Bruno Buchberger (60th birthday).

[2] D. Cheon, *Formalizing faugère's f_4 algorithm in lean 4*, `https://github.com/dongwook0826/FaugereF4` (2026).

[3] J.C. Faugère, *A new efficient algorithm for computing gröbner bases (f4)*, Journal of Pure and Applied Algebra 139 (1999), pp. 61–88. Available at `https://www.sciencedirect.com/science/article/pii/S0022404999000055`.

[4] J. Guo, H. Shen, J. Liu, and L. Zhi, *Formalizing gröbner basis theory in lean* (2026). Available at `https://arxiv.org/abs/2602.12772`.

[5] A. Maletzky, *A generic and executable formalization of signature-based gröbner basis algorithms*, Journal of Symbolic Computation 106 (2021), pp. 23–47. Available at `https://www.sciencedirect.com/science/article/pii/S0747717120301280`.

[6] A. Maletzky and F. Immler, *Gröbner Bases of Modules and Faugère's $$F_4$$Algorithm in Isabelle/HOL*, in *Intelligent Computer Mathematics*, F. Rabe, W.M. Farmer, G.O. Passmore, and A. Youssef, eds., Cham. Springer International Publishing, 2018, pp. 178–193.

[7] The Lean Team, *The Lean Language Reference*, version 4.27.0-rc1 ed. (2026). Available at `https://lean-lang.org/doc/reference/latest/releases/v4.27.0/`, Accessed: 2026-10-26.

[8] The mathlib Community, *The Lean Mathematical Library*, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, Jan., New Orleans, LA, USA. ACM, CPP 2020, 2020. Available at `https://doi.org/10.1145/3372885.3373824`.

[9] L. Théry, *A machine-checked implementation of buchberger's algorithm*, J. Autom. Reasoning 26 (2001), pp. 107–137.